

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА №42

КУРСОВАЯ РАБОТА (ПРОЕКТ)
ЗАЩИЩЕНА С ОЦЕНКОЙ
РУКОВОДИТЕЛЬ

доцент, канд. техн. наук
должность, уч. степень, звание

подпись, дата

О. И. Красильникова
инициалы, фамилия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОЙ РАБОТЕ (ПРОЕКТУ)

по дисциплине:

Web-технологии

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № _____ 4321

подпись, дата

И.А. Лаврешин
инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
1 Дизайн сайта	3
1.1 Модель сайта	4
1.2 Макет сайта	7
1.3 Применение технологии каскадных таблиц стилей.....	10
1.4 Графическое оформление web-страницы.....	15
1.5 Создание навигационных панелей для сайта.....	17
2 Web-сценарии сайта на языке JavaScript	22
2.1 Назначение языка JavaScript.....	22
2.2 Разработка JS-скриптов для сайта.....	22
3 Валидация кода и продвижение сайта	27
3.1 Валидация кода сайта	27
3.2 Способы продвижения сайта.....	28
3.3 Используемые приемы продвижения сайта	31
ЗАКЛЮЧЕНИЕ	33
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	34
ПРИЛОЖЕНИЕ	35

ВВЕДЕНИЕ

Современное развитие цифровой среды требует от веб-студий эффективных инструментов презентации услуг. В рамках курсового проекта разработан веб-ресурс для компании «Сайт?Легко!», ориентированный на удобство пользователя и визуальную целостность. Целью работы стало проектирование и реализация адаптивного сайта-портфолио с интерактивным интерфейсом.

Сайт построен по архитектуре SPA: контент динамически формируется в JavaScript на основе данных из внешних файлов, что упрощает поддержку и расширяемость проекта. В ходе разработки решены ключевые задачи: выполнена адаптивная верстка с отдельной логикой десктопной и мобильной навигации (бургер-меню); реализованы web-сценарии для интерактивных элементов (счетчики, слайдер отзывов); интегрирована форма обратной связи с отправкой данных через Telegram Bot API и валидацией. Юридические разделы вынесены в отдельные модули для обеспечения единого стиля. Результатом работы является готовая платформа, демонстрирующая эффективность data-driven подхода и клиентской логики для быстрого обновления портфолио и функционала.

1 Дизайн сайта

Дизайн сайта «Сайт? Легко!» является ключевым этапом разработки, так как именно он формирует первое впечатление у посетителя и определяет удобство взаимодействия с контентом. Визуальная концепция была ориентирована на создание современного портфолио, где пользователь быстро понимает, кто автор, какие услуги предлагаются, какие проекты уже выполнены, а также каким образом можно отправить заявку. На этапе проектирования были подготовлены макеты в Figma для основных сценариев использования: просмотр главной страницы, переход к портфолио, разделу услуг, блогу, странице контактов с формой, а также юридическим разделам (политика конфиденциальности и пользовательское соглашение). Дополнительно в макетах отражались варианты для разных размеров экрана.

Цветовая схема сайта строится на градиентном фоне и акцентных элементах в сине-фиолетовой гамме. Такой выбор поддерживает узнаваемость бренда, повышает контраст между фоном и карточками контента и делает интерфейс визуально «легче» по восприятию. Для крупного фоновых слоя используется плавная заливка, а для навигационных и информационных блоков применяются светлые поверхности. Визуальная иерархия при этом выстроена через размеры шрифтов, отступы и различия в стилях заголовков.

Макет предусматривает единый скелет страницы: сверху располагается навигация, в центральной части размещается основное содержимое (контент конкретного раздела), а в нижней части поддерживаются элементы завершения интерфейса (социальные ссылки и блоки, дополняющие структуру). Навигация реализована как самостоятельный элемент: на десктопных экранах отображается горизонтальное меню с выделением активного пункта, а на мобильных — используется бургер-меню с выездной панелью и затемнением для фокусировки внимания. В дизайн-проработке заложено, что пользователь должен одинаково легко ориентироваться на любом устройстве: в макетах это прослеживается по разному отображению меню и расположению элементов страницы относительно ширины экрана.

Важная часть дизайнерского решения — карточная компоновка контента. Портфолио оформляется в виде сетки карточек проектов, где присутствует превью, название, краткое описание, при необходимости год и технологический стек. Аналогично услуги размещаются в карточках, что позволяет сравнивать предложения между собой и воспринимать их структуру быстро. Для блога используется карточный список статей с превью изображением, датой, категорией и кнопкой перехода к чтению. Такой подход делает интерфейс логичным: пользователь находит нужный тип информации по визуальному шаблону.

На главной странице дизайн дополняется элементами, повышающими вовлеченность: статистические блоки (с визуальными карточками). Также визуально представлена витрина навыков: прогресс-бары формируют понимание уровня компетенций и дополняют текстовые блоки. Визуальная целостность достигается тем, что все эти элементы выполнены в единой стилистике: одинаковая логика отступов, единые принципы округления элементов и теней, согласованные цвета текста и фона.

Таким образом, дизайн сайта «Сайт?Легко!» отражает баланс между эстетикой и функциональностью: он обеспечивает понятную навигацию, быструю идентификацию разделов, структурирование информации через карточки и сетки, а также адаптацию интерфейса под мобильные устройства. Макеты, разработанные в Figma, фиксируют итоговую композицию страниц и демонстрируют, что один и тот же брендовый стиль сохраняется во всех сценариях использования, включая просмотр портфолио, чтение блога и отправку заявки через форму контактов.

1.1 Модель сайта

Организационная модель сайта построена по принципу приложения SPA, где все ключевые разделы отображаются внутри единой HTML-страницы, а переключение между ними выполняется динамически средствами JavaScript. В отличие от классической многостраничной структуры, в данном

проекте пользователь не переходит на отдельные физические HTML-файлы для каждого раздела. Вместо этого используется центральный контейнер контента (`#content`), который перерисовывается в зависимости от выбранного пункта меню. Такой подход позволяет обеспечить быстрый отклик интерфейса, более плавную навигацию и единый визуальный стиль во всех разделах.

С точки зрения архитектуры модель сайта включает следующие логические уровни:

Уровень каркаса — `index.html`: содержит постоянные элементы интерфейса: верхнюю навигацию (включая мобильное меню), основной контейнер для контента и нижний футер с юридическими ссылками. Этот уровень не меняется при переходах между разделами.

Уровень маршрутизации — `window.updateContent(pageId)` в `script.js`: определяет, какой логический раздел нужно показать пользователю по идентификатору страницы (`home`, `portfolio`, `services`, `blog`, `contacts`, `privacy-policy`, `terms-of-use`).

Уровень контента — файлы `data/*.js`: каждый раздел имеет собственный источник данных (`homeData`, `portfolioData`, `servicesData`, `blogData`, `contactsData` и т.д.), что упрощает поддержку и расширение проекта.

Таким образом, модель сайта представляет собой гибрид “единая оболочка + динамически подгружаемые разделы”.

Описание страниц и их назначения:

Главная: представляет автора и проект – имя, краткая биография, навыки, статистика, преимущества. Это входная точка сайта и основной блок формирования первого впечатления.

Портфолио: содержит карточки выполненных проектов с изображениями, описанием, годом и стеком технологий. Раздел демонстрирует практический опыт и качество работ.

Услуги: описывает перечень услуг (например, лендинг, многостраничный сайт, интернет-магазин), примерные сроки и стоимость. Основная цель — конвертировать интерес пользователя в заявку.

Блог: отображает список статей в карточном формате (дата, категория, превью, статистика). Выполняет функцию контентного продвижения и повышения доверия к автору.

Страницы постов: детальные публикации, открываемые из раздела блога. Включают полный текст статьи, теги, информацию об авторе и элементы шаринга.

Контакты: предоставляет контактную информацию, ссылки на соцсети и форму обратной связи. Через форму пользователь может отправить сообщение напрямую в Telegram.

Политика конфиденциальности: юридический раздел, описывающий порядок обработки персональных данных и правила использования информации, передаваемой через форму.

Пользовательское соглашение: определяет условия использования сайта и ответственность сторон. Повышает юридическую прозрачность ресурса.

Основные связи внутри модели, представленные на рисунке 1, имеют централизованный характер: верхнее меню связывает пользователя с разделами: home, portfolio, services, blog, contacts. Из раздела Блог реализован переход к конкретным постам и возврат назад к списку статей. Из раздела Услуги реализован сценарий перехода в Контакты с автозаполнением темы заявки. Из футера доступны переходы в юридические разделы, а также обратный переход на главную.

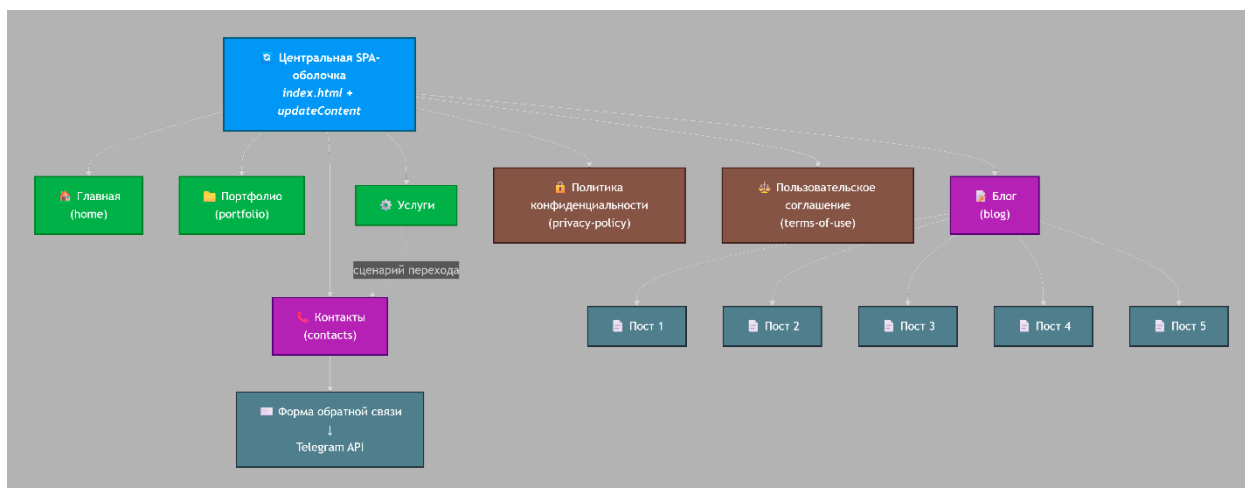


Рисунок 1 — Схема организационной модели сайта

Итоговая модель сайта может быть охарактеризована как иерархически-центрическая SPA-модель: все страницы логически автономны по содержанию, но технически объединены общей оболочкой и единым механизмом маршрутизации. Это обеспечивает простую навигацию, целостный пользовательский опыт и удобство дальнейшего развития проекта.

1.2 Макет сайта

Макет сайта выступает основой всей дальнейшей разработки, потому что именно на этом этапе фиксируется, как пользователь будет воспринимать информацию и как он будет взаимодействовать с интерфейсом. Макет был оформлен в графическом редакторе Figma с упором на создание единого стилового каркаса для всех разделов сайта. Цель состояла в том, чтобы интерфейс оставался понятным и визуально согласованным при переключении между страницами: главная, портфолио, услуги, блог, контакты, а также юридические разделы — политика конфиденциальности и пользовательское соглашение. Благодаря заранее подготовленным композициям в Figma, представленными на рисунке 2, разработка превращается из «перебора вариантов» в контролируемую реализацию конкретной структуры, где каждое изменение уже привязано к утвержденному макету.

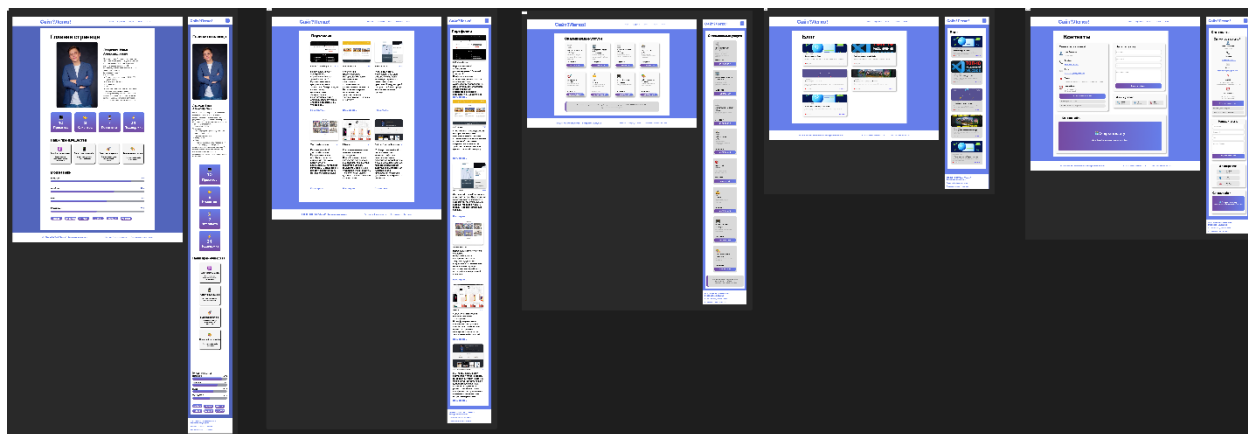


Рисунок 2 — Макет сайта в Figma

На десктопной версии макет ориентирован на горизонтальную навигацию, расположенную в верхней части. Такая компоновка позволяет быстро переключаться между разделами и обеспечивает привычный сценарий использования: пользователь сразу видит пункты меню и может перейти к нужному блоку без лишних шагов. Важным дизайнерским принципом является выделение активного пункта меню: это снижает вероятность ошибки пользователя и делает интерфейс предсказуемым. Макет также предусматривает центральную область контента с ограничением по ширине и выверенными отступами. Это влияет на читабельность текста и корректную сетку карточек: блоки не “расползаются” на больших мониторах и сохраняют аккуратные пропорции на разных разрешениях.

Внутри основного контейнера макет использует модульный подход: ключевые элементы представляют собой самостоятельные блоки, которые в процессе разработки могут собираться в нужной последовательности. На главной странице центральное место занимают элементы, формирующие доверие и ценность автора: приветственный блок с изображением, биография, список навыков, а также статистика и витрина преимуществ. Важной частью макета является карточная структура: преимущества, проекты и услуги в интерфейсе представлены карточками, благодаря чему пользователь может быстро сканировать информацию глазами и сравнивать элементы по схожей структуре.

Макет портфолио разработан с акцентом на визуальные превью и краткие пояснения к проектам. В композиции предусмотрены изображения проектов, заголовки, описание и технологический стек в виде бейджей. Такое решение делает страницу визуально “плотной” по смыслу, но не перегруженной визуально, поскольку текстовая часть ограничивается понятными блоками. Для услуг применён аналогичный принцип карточек: в макете заложены иконка/символ, название, ориентировочный срок выполнения, описание и цена или формулировка “цена договорная”. В блоге карточки статей также собраны в узнаваемый шаблон: изображение, дата/категория, заголовок, краткое превью и кнопка перехода к чтению. В результате во всех разделах повторяется одна логика визуального представления информации, что повышает скорость понимания интерфейса.

Отдельного внимания в макете заслуживает адаптивность. На предоставленных экранах Figma видно, что при переходе к мобильным размерам меняется логика навигации и компоновка контента. Для мобильной версии в макете используется бургер-меню: оно открывает выездную панель слева и затемнение фона (оверлей), чтобы фокусировать внимание пользователя на меню. Важно, что в макете предусмотрена достаточная ширина панели для удобного клика по пунктам, а также аккуратные разделители между ссылками. Само содержимое страницы при этом остаётся доступным после закрытия меню, что соответствует общей логике SPA: пользователь не перезагружает страницу, а переключает разделы внутри одного экрана приложения.

Адаптивная структура также проявляется в изменении компоновки сеток: на узких экранах карточки располагаются более вертикально, и пользователь получает последовательный сценарий прокрутки “сверху вниз”. Это снижает необходимость горизонтального скrolла и улучшает читаемость. В макете отдельно учтены элементы, которые должны оставаться узнаваемыми даже при уменьшении экрана: кнопки, заголовки, карточные блоки, а также форма контактов.

Таким образом, макет сайта решает несколько задач одновременно: формирует единый визуальный стиль, определяет структуру каждого раздела, задаёт карточную логику представления информации и обеспечивает корректную адаптацию под мобильные устройства.

1.3 Применение технологии каскадных таблиц стилей

Для реализации визуальной части сайта применяются каскадные таблицы стилей (CSS), размещённые в едином файле style.css. Использование CSS в данном проекте позволяет разделить задачи: логика формирования страниц вынесена в JavaScript (script.js), а внешний вид и интерактивные состояния управляются стилями. Такой подход упрощает поддержку проекта: изменения дизайна (цвета, отступы, оформление карточек, адаптивность) выполняются без затрагивания структуры и сценариев.

В начале файла реализован базовый глобальный сброс свойств, представленный на рисунке 3: селектор задаёт `margin: 0`, `padding: 0` и `box-sizing: border-box`. Это важно, потому что у разных браузеров и окружений по умолчанию значения полей и внутренних отступов могут отличаться. Применение `box-sizing: border-box` делает поведение размеров элементов более предсказуемым: ширина и высота считаются с учётом рамки и внутреннего содержимого. Дополнительно явно задаются свойства для `html`, `body`: `width: 100%`, `min-height: 100vh`, шрифт интерфейса и общий фон. Фон выполнен градиентом, благодаря чему брендовая гамма визуально объединяет весь сайт.

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

html, body {
  margin: 0;
  padding: 0;
  width: 100%;
  min-height: 100vh;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}
```

Рисунок 3 — Сброс свойств

Дальше стили разделяются по смысловым блокам. Для верхней навигации используется контейнер `.menu` с белой подложкой, тенью `box-shadow` и внутренними отступами. Расположение элементов меню задаётся через `.menu-container`: используется `display: flex`, выравнивание `justify-content: space-between` и `align-items: center`, что обеспечивает равномерное распределение логотипа и навигационных ссылок, что представлено на рисунке 4. Внутри пунктов меню применяется группировка `gap: 2rem`, а активный пункт выделяется изменением цвета и подчеркиванием через `border-bottom-color`. Это визуально помогает пользователю понимать, где он находится в структуре SPA.

```
/* ДЕСКТОПНОЕ МЕНЮ */
.menu {
  background: white;
  padding: 1rem;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
  width: 100%;
}

.menu-container {
  max-width: 1400px;
  margin: 0 auto;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 2rem;
}

.logo {
  font-size: 1.5rem;
  font-weight: bold;
  color: #667eea;
  text-decoration: none;
}

.menu-items {
  display: flex;
  gap: 2rem;
  list-style: none;
}

.menu-items a {
  text-decoration: none;
  color: #333;
  font-weight: 500;
  cursor: pointer;
  padding: 0.5rem 0;
  border-bottom: 2px solid transparent;
}

.menu-items a:hover,
.menu-items a.active {
  color: #667eea;
  border-bottom-color: #667eea;
}
```

Рисунок 4 — Стили для десктопного меню

Для мобильной версии применяются медиазапросы `@media (max-width: 768px)` и `@media (min-width: 769px)`, что позволяет переключать поведение интерфейса без перезагрузки страницы. На ширинах до 768px отображается бургер-кнопка `.burger-btn`, которая фиксируется в правой верхней области

(position: fixed, top/right, высокий z-index). Анимация открытия реализуется чисто средствами CSS: при наличии класса .open у бургер-кнопки изменяются свойства линий .burger-line — первая линия поворачивается и сдвигается, вторая линия исчезает, третья линия поворачивается в противоположную сторону. Затемнение фона делается через .menu-overlay с полу-прозрачным чёрным слоем и backdrop-filter: blur(3px), чтобы пользователь концентрировался на меню. Само меню .mobile-menu — это выездная панель слева, изначально смещённая left: -280px, а при активации возвращающаяся в left: 0 с плавным transition. На десктопе (min-width: 769px) бургер скрывается (display: none), а мобильные элементы меню и оверлей также отключаются.

Помимо навигации CSS активно используется для визуализации интерактивных компонентов. Например, в блоке слайдера заданы .slider-container, .slider-track, .slide и элементы управления. Контейнер слайдера имеет height: 500px, overflow: hidden, чёрный фон и выраженную округлённую форму (border-radius: 20px) с тенью. Плавность переключения обеспечивается transition: transform 0.5s ease у трека .slider-track. Кнопки управления .slider-btn позиционируются абсолютно и стилизуются так, чтобы при наведении менялись фон и цвет текста, а “точки” (.dot, .dot.active) реагировали на состояние активного слайда.

Отдельное внимание уделено оформлению контента и карточек. Для общего содержимого используется контейнер .content с белым фоном, округлением (border-radius: 10px), тенями и фиксированным внутренним ритмом (padding: 2rem). Заголовки и текст имеют типографические правила: h1 окрашивается в тёмный цвет, получает нижнюю границу в фирменной гамме и увеличенный размер, а p задаётся с читаемым размером и межстрочным расстоянием (line-height: 1.6). Таким образом обеспечивается единая типографическая система.

На главной странице CSS формирует сетки для статистики .stats-grid и преимуществ .features-grid. Статистика построена на display, благодаря чему карточки сами адаптируются под ширину. Карточки .stat-card имеют

градиентную подложку и анимацию появления `animation: slideInUp 0.5s ease`. Преимущества оформлены карточками с белым фоном, тенями, скруглениями и эффектом наведения: карточка слегка поднимается (`transform: translateY(-10px)`) и усиливает тень, что добавляет “живости” интерфейсу.

Блок отзывов реализует горизонтальный слайдер через трансформации, а точки `.testimonial-dot` реагируют на активность, представлено на рисунке 5, через `.active` (цвет и масштаб `transform: scale(1.3)`). Визуальная поддержка также прослеживается в кнопках навигации отзыва `.testimonial-btn` — они круговые, с фирменным фоном и абсолютным позиционированием по центру вертикали.

```
.testimonial-btn {
  position: absolute;
  top: 50%;
  transform: translateY(-50%);
  width: 40px;
  height: 40px;
  background: #667eea;
  color: white;
  border: none;
  border-radius: 50%;
  cursor: pointer;
  font-size: 20px;
  z-index: 10;
}

.testimonial-btn.prev { left: 0; }
.testimonial-btn.next { right: 0; }

.testimonial-dots {
  display: flex;
  justify-content: center;
  gap: 10px;
  margin-top: 1rem;
}

.testimonial-dot {
  width: 10px;
  height: 10px;
  border-radius: 50%;
  background: #ccc;
  cursor: pointer;
  transition: 0.3s;
}

.testimonial-dot.active {
  background: #667eea;
  transform: scale(1.3);
}
```

Рисунок 5 — Реализация кнопок и точек слайдера

Для формы обратной связи стили также выполняют роль UX-механизма. Правая колонка `.contacts-right` структурируется через `flex-direction: column` и `gap`. Форма `.contact-form` отображает поля в колонку с единым расстоянием между элементами. Поля и `textarea` имеют одинаковые отступы, закругления и

рамку; при фокусе (:focus) меняется цвет рамки, что даёт пользователю понятный сигнал о текущем вводе, реализация представлена на рисунке 6. Кнопка отправки .submit-btn сделана градиентной, с анимацией наведения: при :hover она слегка перемещается вверх и добавляет тень. В разделе контактов предусмотрен также блок “карта”, у которого появляется затемнение через .map-overlay и backdrop-filter при наведении.

```
/* Форма */
.contact-form-section {
  background: white;
  padding: 30px;
  border-radius: 20px;
  box-shadow: 0 5px 20px rgba(0,0,0,0.05);
}

.contact-form-section h3 {
  color: #333;
  margin-bottom: 20px;
  font-size: 1.5rem;
}

.contact-form {
  display: flex;
  flex-direction: column;
  gap: 15px;
}

.contact-form input,
.contact-form textarea {
  width: 100%;
  padding: 12px 15px;
  border: 2px solid #f0f2f5;
  border-radius: 10px;
  font-size: 1rem;
  transition: border-color 0.3s;
  font-family: inherit;
}

.contact-form input:focus,
.contact-form textarea:focus {
  outline: none;
  border-color: #667eea;
}
```

Рисунок 6 — Анимация контактной формы

Оформление разделов портфолио, услуг, блога и правовых страниц поддерживает единый принцип: карточки и секции имеют border-radius, тени, контролируемые размеры изображений и анимации наведения. Для портфолио используются эффекты transform и масштабирование изображения при наведении, для блога — изменение тени и сдвиг карточки. Юридические страницы .legal-page оформляются по единой схеме: ограничение ширины, цветовая акцентировка заголовков, разделение секций линиями снизу и аккуратный отступный ритм.

Таким образом, применение CSS в проекте обеспечивает не только “красивую картинку”, но и функциональную поддержку интерфейса: адаптивность через @media, визуальные состояния через :hover/:focus, интерактивность через transition и @keyframes, единый стиль карточек и типографики, а также согласованность фирменных цветов по всему сайту. Это делает проект пригодным для демонстрации услуг, портфолио и сбора заявок, сохраняя при этом удобство на разных устройствах.

1.4 Графическое оформление web-страницы

В проекте графика используется как функциональный и композиционный инструмент, основная цель которого — повысить читаемость интерфейса, ускорить восприятие контента и обеспечить визуальную целостность SPA-приложения при переходах между разделами. В текущей реализации графические элементы применяются в нескольких ролях. Это фоновая графика в виде градиента для html и body, которая формирует фирменный стиль. Это иллюстрации и превью в карточках портфолио и блога для визуального акцентирования контента. Это изображения в карточках и блоках разделов как элементы навигационного и смыслового сопровождения. И наконец, это декоративные графические эффекты, такие как тени, градиентные подложки и hover-анимации, которые усиливают интерактивность интерфейса.

Для проекта выбраны форматы растровой графики JPG/PNG, размещенные в каталоге images, что представлено на рисунке 7.

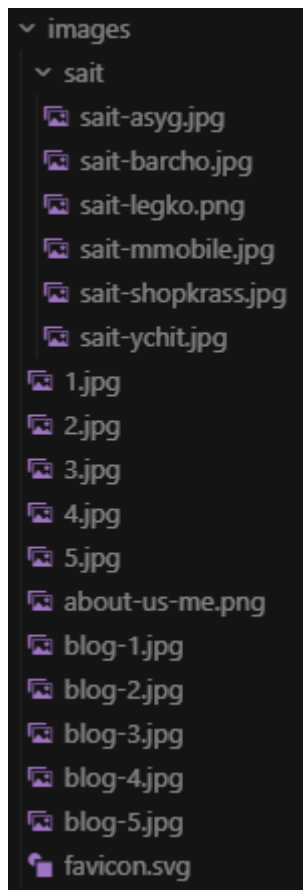


Рисунок 7 — Медиафайлы в папке images

Выбор форматов обусловлен характером контента. JPG применяется для фотографий и сложных изображений с множеством цветовых переходов, так как он оптимален по соотношению качество вес. PNG используется там, где важна четкость деталей или требуется прозрачность, например для элементов интерфейса и отдельных иллюстраций. Такой подход позволяет получить приемлемое качество изображения при контролируемом размере файлов. Вопросы оптимизации графики в проекте решаются на уровне структуры данных, вёрстки и стилей. Во-первых, изображения подключаются по мере рендеринга раздела, что соответствует логике SPA и исключает вывод лишней графики. Во-вторых, реализована обработка неполных данных: при отсутствии изображения используется fallback, то есть заглушка, либо блок не выводится, благодаря чему не нарушается компоновка страницы. В-третьих, для стабильного внешнего вида карточек используется единый шаблон отображения с фиксированными контейнерами и параметрами object-fit: cover

и `object-position: center`, что исключает скачки сетки из-за разных пропорций исходных файлов. Дополнительно для ключевых изображений заданы атрибуты `alt`, что повышает доступность и улучшает SEO-представление страницы.

Также в проект добавлена работа с векторной графикой SVG: иконка сайта (favicon) реализована в формате SVG и подключена в `head` через `<link rel="icon" type="image/svg+xml" href="favicon.svg">`. Использование SVG для favicon дает ряд преимуществ: масштабирование без потери качества на любых разрешениях экрана, небольшой размер файла и удобство редактирования (цвета, форма и композиция могут корректироваться напрямую в коде). Это дополняет растровую графику и усиливает визуальную целостность интерфейса. Иконка сайта представлена на рисунке 8.



Рисунок 8 — Иконка сайта (favicon)

С точки зрения оформления страницы применены современные приемы фоновой и декоративной графики: единый градиентный фон, карточная композиция, скругления, тени и плавные переходы при наведении. Эти решения визуально объединяют все разделы, поддерживают консистентность интерфейса и делают взаимодействие с сайтом более понятным для пользователя.

1.5 Создание навигационных панелей для сайта

Навигационная система сайта является одним из ключевых элементов, определяющих удобство пользовательского опыта. Так как проект реализован как SPA, пользователь ожидает, что переход между разделами будет происходить быстро и без перезагрузки страницы. Поэтому навигационные

панели строятся так, чтобы управление контентом осуществлялось не средствами стандартного перехода по ссылкам, а программной логикой: при клике обновляется отображаемая секция (через функцию `window.updateContent`), а интерфейс сохраняет визуальную целостность. В проекте предусмотрены две формы навигации: десктопное верхнее меню и мобильное бургер-меню с выездной панелью и оверлеем.

Десктопная навигация реализована через React-компонент `Menu`, встроенный прямо в `index.html`. Внутри компонента формируется массив пунктов меню `menuItems`, содержащий идентификаторы разделов. Далее используется состояние `activeItem`, которое хранит текущий активный пункт. С точки зрения интерфейса это важно, потому что пользователь должен визуально понимать, где он находится в структуре сайта. При рендеринге ссылок меню каждому пункту назначается класс `active`, если `activeItem === item.id`. Такое выделение реализовано средствами CSS: для `.menu-items a.active` меняется цвет и появляется акцентная нижняя граница, что делает текущий раздел различимым даже при быстром сканировании экрана, пример представлен на рисунке 9.

Сайт?Легко!

Главная Портфолио Услуги Блог Контакты

Рисунок 9 — Пример активного пункта меню

Обработка клика по пункту меню построена на функции `handleClick`. При выборе раздела происходит предотвращение стандартного поведения ссылки (`e.preventDefault()`), затем активный пункт меняется через `setActiveItem(itemId)`, после чего вызывается `closeMenu()`. Эта процедура специально отвечает за закрытие мобильного меню: она снимает класс `.active` у выездной панели и оверлея, а также убирает класс `.open` у кнопки бургера. После этого, если доступна функция `window.updateContent`, компонент вызывает её с выбранным `itemId`. Таким образом, навигация не делает полноценную перезагрузку документа: контент переключается внутри одной

страницы, а текущее состояние интерфейса (меню, стили, плавность перехода) сохраняется.

Мобильная навигация реализована как адаптивное продолжение той же логики пунктов меню. В `index.html` присутствуют элементы `mobileMenu` и `menuOverlay`, которые отвечают за выездную панель слева и затемнение фона при открытии, они представлены на рисунке 10. При открытии мобильного меню бургер-кнопка меняет классы у панели и оверлея: в обработчике `onClick` выполняется `classList.toggle('active')` для `mobileMenu` и `menuOverlay`, а также `classList.toggle('open')` для самого бургер-элемента. Дополнительно применяются CSS-анимации: в `style.css` при наличии `.burger-btn.open` линии бургера поворачиваются и складываются в “крестик” (через `transform` и `opacity`), что визуально показывает пользователю состояние меню. Оверлей (`.menu-overlay`) создаёт эффект фокусировки: на мобильных устройствах он отображается как полу-прозрачный слой с `backdrop-filter`, и за счёт этого выездная панель воспринимается как отдельный модальный слой.

```

<script type="text/babel">
  const closeMenu = () => {
    document.getElementById('mobileMenu').classList.remove('active');
    document.getElementById('menuOverlay').classList.remove('active');
    document.querySelector('.burger-btn').classList.remove('open');
  };
  const Menu = () => {
    const [activeItem, setActiveItem] = React.useState('home');

    const menuItems = [
      { id: 'home', label: 'Главная' },
      { id: 'portfolio', label: 'Портфолио' },
      { id: 'services', label: 'Услуги' },
      { id: 'blog', label: 'Блог' },
      { id: 'contacts', label: 'Контакты' }
    ];

    const handleClick = (itemId, e) => {
      e.preventDefault();
      setActiveItem(itemId);
      closeMenu();
      if (window.updateContent) {
        window.updateContent(itemId);
      }
    };

    React.useEffect(() => {
      const mobileMenu = document.getElementById('mobileMenuItems');
      if (mobileMenu) {
        mobileMenu.innerHTML = '';
        menuItems.forEach(item => {
          const li = document.createElement('li');
          const a = document.createElement('a');
          a.href = '#';
          a.textContent = item.label;
          a.onclick = (e) => handleClick(item.id, e);
          li.appendChild(a);
          mobileMenu.appendChild(li);
        });
      }
    }, []);

    return (
      <>
        { /* кнопка для открытия/закрытия мобильного меню */ }
        <button className="burger-btn" id="burgerBtn" onClick={() => {
          document.getElementById('mobileMenu').classList.toggle('active');
          document.getElementById('menuOverlay').classList.toggle('active');
          document.getElementById('burgerBtn').classList.toggle('open');
        }}>
          <span className="burger-line"></span>
          <span className="burger-line"></span>
          <span className="burger-line"></span>
        </button>

        { /* десктоп меню */ }
        <nav className="menu">
          <div className="menu-container">
            <a href="/" className="logo" onClick={(e) => handleClick('home', e)}>
              Сайт?lerko!
            </a>
            <ul className="menu-items">
              {menuItems.map(item => (
                <li key={item.id}>
                  <a
                    href="#"
                    className={activeItem === item.id ? 'active' : ''}
                    onClick={(e) => handleClick(item.id, e)}>
                      {item.label}
                    </a>
                </li>
              ))}
            </ul>
          </div>
        </nav>
      </>
    );
  };
</script>

```

Рисунок 10 — Реализация меню

Заккрытие мобильного меню реализовано не только кликом по пункту, но и кликом по затемненному фону. В index.html предусмотрена строка `document.getElementById('menuOverlay').onclick = closeMenu;`, что соответствует ожидаемому паттерну: пользователь может отменить выбор, нажав на фон. Это повышает удобство и снижает количество действий для возврата к контенту.

Непосредственное переключение разделов организовано в `script.js`. Изначально функция `window.updateContent` берёт элемент `#content` и заменяет его внутренний HTML на результат `buildPage(pageId)`. Таким образом, вся навигация фактически управляет параметром `pageId`. Важно, что при выборе `home` дополнительно запускается анимация статистики (`animateNumbers`), что делает переход в главную не просто визуальным переключением, а “живым” сценарием.

В итоге навигация сайта спроектирована как связка “меню (React) + контентный рендер (`script.js`) + стили (CSS)”. React отвечает за управление состоянием активного пункта и адаптивную генерацию пунктов мобильного меню, CSS обеспечивает корректное визуальное поведение (выделение активного пункта, анимация бургера, оверлей и выездная панель), а JavaScript реализует фактическое переключение контента через `pageId`. Такой подход обеспечивает предсказуемость для пользователя, скорость работы и единый визуальный стиль на всех устройствах.

2 Web-сценарии сайта на языке JavaScript

2.1 Назначение языка JavaScript

Назначение JavaScript в проекте — обеспечение клиентской динамики и интерактивности. Язык выступает основным инструментом реализации сайта SPA: он управляет маршрутизацией через программное переключение контента, подгружая данные из глобальных объектов, и отвечает за построение страниц без перезагрузки. Кроме того, JavaScript реализует интерактивные элементы — анимированную статистику, слайдер отзывов с переключением состояний, обработку форм с отправкой данных через Telegram Bot API, а также сценарии блога (открытие постов, копирование ссылок, адаптивное использование Web Share API). Дополнительно скрипты управляют UX-улучшениями: плавной прокруткой, автоматическим заполнением полей и подсветкой элементов. В совокупности JavaScript превращает сайт из набора статичных страниц в современное интерактивное приложение.

2.2 Разработка JS-скриптов для сайта

Разработка JS-скриптов для сайта направлена на то, чтобы проект воспринимался как полноценное приложение (SPA), где пользовательские переходы, отрисовка контента и интерактивные элементы происходят без перезагрузки страницы. Вся клиентская логика сосредоточена в `script.js`, а данные для страниц вынесены в отдельные файлы `data/*.js`. Такой подход упрощает сопровождение: структура страницы и её внешний вид задаются CSS, а JavaScript занимается сборкой контента и управлением сценариями.

Одной из центральных задач разработки JS является создание универсального механизма рендера страниц. В проекте для этого используется функция `buildPage(pageId)`, представленная на рисунке 11. Она принимает идентификатор текущего раздела и на его основе формирует HTML-страницу: от заголовка до конкретных блоков. Внутри `buildPage` происходит выбор источника данных. Для основных разделов применена стандартная схема: `data`

= window[pageId + 'Data']. Для страниц юридической тематики (privacy-policy и terms-of-use) предусмотрены явные ветки, где данные берутся из window.privacyPolicyData и window.termsOfUseData. Если нужные данные не найдены, возвращается сообщение об ошибке загрузки — это служит дополнительной устойчивости проекта при расширении.

Далее внутри buildPage формируются блоки каждой страницы. Для главной страницы собираются элементы, основанные на данных homeData: приветственный контент, секция навыков, текст про проекты, а также динамические элементы. JavaScript влияет на то, как будет выглядеть компонент уже в момент генерации разметки.


```

function buildPage(pageId) {
  let data;
  if (pageId === 'privacy-policy') {
    data = window.privacyPolicyData;
  } else if (pageId === 'terms-of-use') {
    data = window.termsOfUseData;
  } else {
    data = window[pageId + 'Data'];
  }

  if (!data) {
    return `<h1>Ошибка загрузки</h1><p>Данные для страницы "${pageId}" не найдены</p>`;
  }

  let html = `<h1>${data.title || ''}</h1>`;

  if (pageId === 'home') {
    html += `
    <div class="home-page">
      <div class="home-grid">
        ${data.image ? `
        <div class="home-image">
          
        </div>
        ` : ''}

        <div class="home-content">
          ${data.name ? `<h2>${data.name}</h2>` : ''}

          ${data.bio || ''}

          <!-- ===== ИСПРАВЛЕНО: используем data.skills ===== -->
          ${data.skills ? `
          <div class="skills-section">
            <h3>Эти роды я освоил:</h3>
            <ul class="skills-list">
              ${data.skills.map(skill => `<li>${skill}</li>`).join('')}
            </ul>
          </div>
          ` : ''}
          <!-- ===== -->

          ${data.projectsText || ''}
        </div>
      </div>

      ${data.stats ? createStats(data.stats) : ''}
      ${data.features ? createFeatures(data.features) : ''}
      ${data.skillsProgress ? createSkills(data.skillsProgress) : ''}
      ${data.techStack ? createTechStack(data.techStack) : ''}
    </div>
    `;
  } else if (pageId === 'portfolio') {
    html += `
    <div class="portfolio-page">
      <div class="portfolio-grid">
        ${data.projects ? data.projects.map(project => `
        <div class="portfolio-card">
          <div class="portfolio-image">
            
          </div>
          <div class="portfolio-info">
            <h3>${project.name}</h3>
            <p class="portfolio-year">${project.year || ''}</p>
            <p class="portfolio-desc">${project.description || ''}</p>
            <div class="portfolio-tech">
              ${project.technologies ? project.technologies.map(tech =>

```

Рисунок 11 — Реализация части функции buildPage

Для интерактивности на главной странице предусмотрена логика анимации числовых счетчиков. Запуск анимации делается при переключении на главную страницу: в window.updateContent используется setTimeout(animateNumbers, 500), то есть анимация начинается после подстановки контента и краткой паузы для корректного появления DOM.

Помимо главной страницы, в buildPage описаны сценарии для портфолио, услуг, блога и контактов. Для портфолио JavaScript перебирает массив проектов и формирует сетку карточек, где подставляются изображение, название, описание, год, список технологий и ссылка

“Посмотреть”. Услуги собираются из `data.items`: для каждой услуги выводятся карточка с иконкой, временем, описанием, ценой и кнопкой. Кнопка “Заказать сайт” вызывает `window.orderService(serviceName)`, тем самым JS связывает раздел услуг с контактной формой. Блог формируется как набор карточек постов: используются данные `data.posts`, и каждая карточка вызывает `window.showPost(index + 1)`.

Наконец, JS отвечает за коммуникацию через форму на контактах, что представлено на рисунке 12. Обработчик `window.handleFormSubmit` перехватывает отправку формы (`event.preventDefault()`), собирает поля (имя, email, тема, сообщение) и отправляет их через `fetch` на Telegram Bot API (`/sendMessage`). После успешной отправки форма очищается и показывается уведомление; при ошибке выводятся сообщения пользователю. Это превращает страницу “Контакты” из статичного макета в рабочий инструмент получения заявок.

```

function buildPage(pageId) {
  let data;
  if (pageId === 'privacy-policy') {
    data = window.privacyPolicyData;
  } else if (pageId === 'terms-of-use') {
    data = window.termsOfUseData;
  } else {
    data = window[pageId + 'Data'];
  }

  if (!data) {
    return `<h1>Ошибка загрузки</h1><p>Данные для страницы "${pageId}" не найдены</p>`;
  }

  let html = `<h1>${data.title || ''}</h1>`;

  if (pageId === 'home') {
    html += `
    <div class="home-page">
      <div class="home-grid">
        ${data.image ? `
          <div class="home-image">
            
          </div>
        ` : ''}

        <div class="home-content">
          ${data.name ? `<h2>${data.name}</h2>` : ''}

          ${data.bio || ''}

          <!-- ===== ИСПРАВЛЕНО: используем data.skills ===== -->
          ${data.skills ? `
            <div class="skills-section">
              <h3>Эти годы я освоил:</h3>
              <ul class="skills-list">
                ${data.skills.map(skill => `<li>${skill}</li>`).join('')}
              </ul>
            </div>
          ` : ''}
          <!-- ===== -->

          ${data.projectsText || ''}
        </div>
      </div>

      ${data.stats ? createStats(data.stats) : ''}
      ${data.features ? createFeatures(data.features) : ''}
      ${data.skillsProgress ? createSkills(data.skillsProgress) : ''}
      ${data.techStack ? createTechStack(data.techStack) : ''}
    </div>
  `;
  } else if (pageId === 'portfolio') {
    html += `
    <div class="portfolio-page">
      <div class="portfolio-grid">
        ${data.projects ? data.projects.map(project => `
          <div class="portfolio-card">
            <div class="portfolio-image">
              
            </div>
            <div class="portfolio-info">
              <h3>${project.name}</h3>
              <p class="portfolio-year">${project.year || ''}</p>
              <p class="portfolio-desc">${project.description || ''}</p>
              <div class="portfolio-tech">
                ${project.technologies ? project.technologies.map(tech =>

```

Рисунок 12 — Форма на странице контакты

В целом разработка JS-скриптов в проекте решает три главные задачи: построение контента по `pageId`, обеспечение интерактивности (анимации, слайдер, динамические формы и кнопки) и интеграция сетевых сценариев (Telegram, share).

3 Валидация кода и продвижение сайта

3.1 Валидация кода сайта

Валидация кода в проекте представляет собой комплексный подход к обеспечению стабильности интерфейса через раннее выявление ошибок и ограничение некорректных действий пользователя. Поскольку проект реализован на чистом HTML/CSS/JavaScript без серверной части и сборочного пайплайна, качество и надежность поддерживаются средствами самой клиентской логики.

Первый уровень валидации обеспечивается встроенными механизмами HTML: обязательные поля формы отмечены атрибутом `required`, а поле `email` использует тип `type="email"`, что заставляет браузер проверять корректность формата до отправки запроса. Это отсеивает невалидные данные на самом раннем этапе. Весь HTML-код успешно прошел валидацию, что подтверждается рисунком 13.



Рисунок 13 — Результат валидации HTML кода

Второй уровень — контроль целостности данных при построении страниц. Функция `buildPage(pageId)` проверяет наличие источника данных (`window[pageId + 'Data']` и специальных веток для юридических страниц), и в случае отсутствия возвращает понятное сообщение об ошибке вместо некорректного HTML. Приложение не “падает молча”, а информирует о проблеме.

Третий уровень связан с защитой интерактивных компонентов. В слайдере отзывов перед изменением индекса выполняется проверка длины массива (`if (testimonials.length === 0) return;`), что предотвращает обращение к несуществующим элементам. Аналогично функции генерации HTML возвращают пустую строку при отсутствии или пустоте входных данных.

Четвертый уровень — контроль сетевых операций. При отправке формы в Telegram используется конструкция `try/catch` для перехвата ошибок соединения, а проверка `response.ok` позволяет корректно отреагировать на успешный или неуспешный ответ. В обоих случаях пользователь получает явное уведомление, а форма при успехе очищается.

Пятая линия — устойчивость сценариев, таких как кнопка “Поделиться” в блоге. Реализована проверка доступности Web Share API с корректным `fallback` через подтверждение, копирование ссылки или открытие Telegram-ссылки, что гарантирует работоспособность функции в любом окружении.

Таким образом, валидация в проекте строится как сочетание встроенных браузерных проверок, защитных механизмов при сборке контента, контроля граничных состояний компонентов, обработки ошибок сетевых запросов и адаптивного поведения сценариев. Это обеспечивает стабильность интерфейса и позволяет выявлять проблемы на этапе до или во время пользовательских действий, а не после отображения некорректного состояния.

3.2 Способы продвижения сайта

Эффективность сайта во многом определяется его заметностью для потенциальных клиентов и способностью удерживать внимание посетителей, что достигается за счет продуманной системы продвижения. Ключевое место в этой системе занимает контентное наполнение, базирующееся на демонстрации реальных проектов, экспертности исполнителя и формировании доверия через отзывы. Стратегия строится вокруг соответствия материалов интересам целевой аудитории, что обеспечивается тематической структурой блога со статьями по востребованным направлениям веб-разработки. Не менее

значимым фактором выступает техническая готовность ресурса: поисковые алгоритмы учитывают корректное отображение на любых устройствах, адаптивность интерфейса, стабильность клиентской логики и скорость загрузки страниц. Дополнительным каналом привлечения аудитории служат встроенные инструменты распространения контента через мессенджеры и социальные платформы, позволяющие наращивать охват как органически, так и с применением платного трафика.

В практике веб-проектов применяются базовые и наиболее результативные способы продвижения: SEO, контент-маркетинг, SMM, email-маркетинг, контекстная и таргетированная реклама, партнерские размещения, а также веб-аналитика для постоянной оптимизации. При этом центральным долгосрочным инструментом выступает SEO-оптимизация, поскольку именно она обеспечивает устойчивый поток целевых пользователей из поисковых систем без прямой зависимости от ежедневных рекламных затрат.

SEO включает несколько взаимосвязанных уровней. Первый уровень — внутренняя оптимизация: формирование семантического ядра, кластеризация запросов по страницам, настройка мета-тегов (title, description), корректная иерархия заголовков (H1–H3), создание человеко-понятных URL, внутренняя перелинковка и повышение качества текстов под поисковый интент. Каждая страница должна решать конкретную задачу пользователя (информационную, навигационную или коммерческую), быть уникальной по содержанию и содержать релевантные ключевые фразы без переоптимизации.

Второй уровень — техническое SEO. Ключевыми параметрами являются скорость загрузки, мобильная адаптивность, корректная индексация страниц, отсутствие битых ссылок, настройка редиректов, наличие robots.txt и sitemap.xml, корректное использование canonical, а также структурированная разметка для улучшения представления сайта в поисковой выдаче. Для SPA-проектов дополнительно критична доступность контента для поисковых роботов: корректная маршрутизация, отсутствие «пустых» страниц при обходе и предсказуемая загрузка контента.

Третий уровень — внешнее SEO: получение качественных тематических ссылок, упоминаний бренда, публикаций на профильных площадках и работа с репутацией. Поисковые системы оценивают не только количество ссылок, но и их качество, тематическую близость и естественность ссылочного профиля. Поэтому приоритет отдается экспертным публикациям и партнерским материалам.

Отдельное значение имеет локальное SEO, особенно для услуг в конкретном регионе. Оно включает оптимизацию геозависимых запросов, заполнение карточек компании в картах и справочниках, единообразие контактных данных, сбор и обработку отзывов, публикацию локального контента и контроль региональной релевантности страниц. Это повышает видимость сайта в локальной выдаче и напрямую влияет на количество заявок.

Контент-маркетинг как базовый способ продвижения поддерживает SEO и усиливает охват. Экспертные статьи, кейсы, разборы проектов, инструкции и FAQ закрывают информационные потребности аудитории, повышают доверие и улучшают поведенческие метрики (время на сайте, глубина просмотра, доля повторных визитов). Дополнительно могут использоваться email-рассылки с новыми публикациями, предложениями и образовательными материалами, что повышает удержание текущей аудитории и стимулирует возврат пользователей на сайт.

К платным способам продвижения относятся контекстная реклама в поисковых системах и таргетированная реклама в социальных сетях. Эти каналы позволяют быстро привлекать целевой трафик, тестировать гипотезы, сегменты аудитории и рекламные сообщения, а также усиливать уже работающие SEO-страницы. В зависимости от задач применяются медийная реклама, ретаргетинг, look-alike аудитории и размещения на профильных площадках.

Существенный вклад в продвижение вносят SMM и репутационный маркетинг. Публикация кейсов, визуальных материалов и экспертных постов в социальных сетях расширяет охват, а работа с комментариями, отзывами и

пользовательским контентом усиливает доверие к бренду. Дополнительный эффект дают партнерские интеграции, гостевые публикации и PR-активности, повышающие узнаваемость и авторитет проекта.

Для оценки результативности продвижения необходима веб-аналитика: отслеживание источников трафика, позиций по ключевым запросам, CTR в поисковой выдаче, поведенческих метрик, конверсий, стоимости привлечения клиента (CAC) и возврата инвестиций (ROMI). На основе этих данных корректируются семантика, контент-план, рекламные кампании, структура сайта и пользовательские сценарии. Таким образом, максимальную эффективность обеспечивает комплексный подход, при котором SEO, контент, социальные каналы, платная реклама и аналитика работают как единая система продвижения, обеспечивающая как быстрый краткосрочный результат, так и устойчивый долгосрочный рост.

3.3 Используемые приемы продвижения сайта

Продвижение сайта в проекте реализовано как совокупность технических, контентных и UX-решений, направленных на повышение удобства, доверия и вероятности целевого действия: при отсутствии серверной части акцент сделан на факторах, которые улучшают поведение пользователей и качество представления ресурса в поиске и соцсетях.

Контент структурирован через понятную иерархию заголовков (h1–h3) и карточную подачу, благодаря чему страницы легче воспринимаются и сканируются; портфолио, услуги и блог оформлены в виде информативных карточек с изображениями, описаниями и метаданными. Дополнительно усилена SEO-оптимизация главной страницы: обновлён title, добавлены мета-теги description, keywords, author, robots, canonical, а также Open Graph (og:type, og:locale, og:site_name, og:title, og:description, og:url) и Twitter Card (twitter:card, twitter:title, twitter:description) для корректного превью ссылок и роста релевантности в выдаче.

Важную роль играет работа с изображениями: для ключевых медиаэлементов используются осмысленные alt-атрибуты, пример представлен на рисунке 14, что повышает доступность и семантическое качество страниц, а блогерская стратегия (даты, категории, теги, полноценные страницы статей) поддерживает содержательное развитие сайта и вовлечённость аудитории.

```
${post.image ? `` : ''}
```

Рисунок 14 — Форма на странице контакты

Распространению материалов способствует кнопка “Поделиться” на базе Web Share API с fallback-сценариями, а наличие социальных каналов, контактных ссылок и интерактивной карты сокращает путь пользователя к обращению. Поведенческие показатели усиливаются за счёт анимаций, слайдера отзывов и hover-эффектов, а доверие повышается наличием юридических разделов в футере. В результате продвижение строится на связке качественного контента, базовой SEO-настройки, корректной социальной разметки, удобной навигации и интерактивности, что повышает вовлечённость и конверсионный потенциал сайта.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была разработана современная веб-страница SPA для демонстрации портфолио, услуг и организации обратной связи. Итоговый продукт включает главную страницу, портфолио, услуги, блог, контакты и юридические разделы.

Архитектура SPA реализована через функции `updateContent(pageId)` и `buildPage(pageId)`, подгружающие данные из модулей `data/*.js`, что обеспечивает разделение контента и логики, упрощая поддержку проекта.

Дизайн проработан в Figma с учётом адаптивности: медиазапросы и бургер-меню обеспечивают корректное отображение на мобильных и десктопных устройствах. Единая стилистика с градиентами, карточками и визуальной иерархией формирует аккуратный интерфейс.

JavaScript выступает логическим центром приложения: отвечает за сборку страниц, анимированную статистику, слайдер отзывов, отправку форм через Telegram Bot API и кнопку «Поделиться» с fallback-сценариями. Интерфейс снабжён hover-эффектами, подсветкой активных элементов и встроенной валидацией формы.

В результате создан адаптивный, функциональный сайт-портфолио, готовый к демонстрации и использованию в качестве инструмента привлечения клиентов. Получен практический опыт проектирования интерфейса, вёрстки и реализации клиентской логики на JavaScript.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГУАП, документация для учебного процесса. – URL: <https://guap.ru/regdocs/docs/uch> (дата обращения 09.03.2026)
2. Каскадные таблицы стилей, анимации, селекторы состояний и макеты – URL: <https://developer.mozilla.org/ru/docs/Web/CSS> (дата обращения 015.02.2026)
3. Документация по хукам `useState` и `useEffect` – URL: <https://react.dev/learn> (дата обращения 18.02.2026)
4. Документация по созданию макетов и компонентной структуре – URL: <https://www.figma.com/resources/> (дата обращения 24.02.2026)
5. Флэнаган, В. JavaScript полное руководство: Учебник / Дэвид Флэнаган. — Диалектика-СПб, 2021. — 689 с
6. Бхаргава, А. Грокаем алгоритмы. — Питер, 2017. — 288 с
7. W3C / спецификация HTML – URL: <https://www.w3.org/TR/html/> (дата обращения 03.03.2026)
8. Официальное руководство по Git и GitHub – URL: <https://git-scm.com/doc> (дата обращения 06.03.2026)

ПРИЛОЖЕНИЕ

Исходный код программы

```
// =====  
// File: index.html  
// =====
```

```
<!DOCTYPE html>  
<html lang="ru">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Мой сайт</title>  
  <link rel="stylesheet" href="style.css">  
  
  <script crossorigin src="https://unpkg.com/react@18/umd/react.development.js"></script>  
  <script crossorigin src="https://unpkg.com/react-dom@18/umd/react-  
dom.development.js"></script>  
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>  
</head>  
<body>  
  <!-- затемнение фона -->  
  <div class="menu-overlay" id="menuOverlay"></div>  
  
  <!-- бургер меню -->  
  <div class="mobile-menu" id="mobileMenu">  
    <div class="mobile-menu-header">Сайт?Легко!</div>  
    <ul class="mobile-menu-items" id="mobileMenuItems">  
      <!-- Пункты будут добавлены через React -->  
    </ul>  
  </div>  
  
  <!-- десктоп меню -->  
  <div id="root"></div>  
  
  <div class="main-container">  
    <div id="content" class="content">  
      <h1>Загрузка...</h1>  
    </div>  
  </div>  
  
  <script src="data/home.js"></script>  
  <script src="data/portfolio.js"></script>  
  <script src="data/services.js"></script>  
  <script src="data/blog.js"></script>  
  <script src="data/contacts.js"></script>
```

```

<script src="data/post1.js"></script>
<script src="data/post2.js"></script>
<script src="data/post3.js"></script>
<script src="data/post4.js"></script>
<script src="data/post5.js"></script>

<script src="data/privacy-policy.js"></script>
<script src="data/terms-of-use.js"></script>

<!-- отвечает только за верхнее меню -->
<script type="text/babel">
  // закрытие мобильного меню и оверлея
  const closeMenu = () => {
    document.getElementById('mobileMenu').classList.remove('active');
    document.getElementById('menuOverlay').classList.remove('active');
    document.querySelector('.burger-btn')?.classList.remove('open');
  };

  // бургер + десктоп навигация
  const Menu = () => {
    const [activeItem, setActiveItem] = React.useState('home');

    const menuItems = [
      { id: 'home', label: 'Главная' },
      { id: 'portfolio', label: 'Портфолио' },
      { id: 'services', label: 'Услуги' },
      { id: 'blog', label: 'Блог' },
      { id: 'contacts', label: 'Контакты' }
    ];

    const handleClick = (itemId, e) => {
      e.preventDefault();
      setActiveItem(itemId);
      closeMenu();
      if (window.updateContent) {
        window.updateContent(itemId);
      }
    };

    // синхр бургер меню с тем же списком пунктов
    React.useEffect(() => {
      const mobileMenu = document.getElementById('mobileMenuItems');
      if(mobileMenu) {
        mobileMenu.innerHTML = "";
        menuItems.forEach(item => {
          const li = document.createElement('li');
          const a = document.createElement('a');
          a.href = '#';
          a.textContent = item.label;

```

```

        a.onclick = (e) => handleClick(item.id, e);
        li.appendChild(a);
        mobileMenu.appendChild(li);
    });
}
}, []);

return (
    <>
        { /* кнопка для открытия/закрытия мобильного меню */ }
        <button className="burger-btn" id="burgerBtn" onClick={() => {
            document.getElementById('mobileMenu').classList.toggle('active');
            document.getElementById('menuOverlay').classList.toggle('active');
            document.getElementById('burgerBtn').classList.toggle('open');
        }} >
            <span className="burger-line"></span>
            <span className="burger-line"></span>
            <span className="burger-line"></span>
        </button>

        { /* десктоп меню */ }
        <nav className="menu">
            <div className="menu-container">
                <a href="/" className="logo" onClick={(e) => handleClick('home', e)} >
                    Сайт?Легко!
                </a>
                <ul className="menu-items">
                    {menuItems.map(item => (
                        <li key={item.id}>
                            <a
                                href="#"
                                className={activeItem === item.id ? 'active' : ''}
                                onClick={(e) => handleClick(item.id, e)}
                            >
                                {item.label}
                            </a>
                        </li>
                    ))}
                </ul>
            </div>
        </nav>
    </>
);
};

// закрытие меню по клику на затемнённый фон
document.getElementById('menuOverlay').onclick = closeMenu;

// react-меню в корневой контейнер
ReactDOM.createRoot(document.getElementById('root')).render(<Menu />);

```

```

</script>
<footer class="footer">
  <div class="footer-container">
    <div class="footer-content">
      <p>© 2026 - ООО "Сайт?Легко!" - Все права защищены.</p>
      <div class="footer-links">
        <a href="#" onclick="window.updateContent('privacy-policy'); return
false;">Политика Конфиденциальности</a>
        <span class="footer-separator">|</span>
        <a href="#" onclick="window.updateContent('terms-of-use'); return
false;">Пользовательское Соглашение</a>
      </div>
    </div>
  </div>
</footer>
<!-- Отправка в Telegram через скрытый iframe (обходит CORS; fetch с api.telegram.org
из браузера часто блокируется) -->
<iframe name="telegramSubmitFrame" id="telegramSubmitFrame" title="Telegram"
style="width:0;height:0;border:0;visibility:hidden;position:absolute" aria-
hidden="true"></iframe>
<script src="script.js"></script>
</body>
</html>

```

```

// =====
// File: script.js
// =====

```

```

// счетчики на главной странице
function createStats(stats) {
  if (!stats || !stats.length) return "";

  return `
    <div class="stats-grid">
      ${stats.map(stat => `
        <div class="stat-card">
          <div class="stat-icon">${stat.icon}</div>
          <div class="stat-value" data-target="${stat.value}">0</div>
          <div class="stat-label">${stat.label}</div>
        </div>
      `).join("")}
    </div>
  `;
}

// карточки преимуществ
function createFeatures(features) {
  if (!features || !features.length) return "";

```

```

return `
  <div class="features-section">
    <h2>Наши преимущества</h2>
    <div class="features-grid">
      ${features.map(f => `
        <div class="feature-card">
          <div class="feature-icon">${f.icon}</div>
          <h3>${f.title}</h3>
          <p>${f.desc}</p>
        </div>
      `).join("")}
    </div>
  </div>
`;
}

// ВСПОМОГАТЕЛЬНЫЙ РЕНДЕР СЛАЙДЕРА ОТЗЫВОВ
function renderTestimonialsInner(testimonials) {
  return `
    <div class="testimonials-slider">
      <button class="testimonial-btn prev" onclick="prevTestimonial()">⏪</button>
      <div class="testimonials-container" id="testimonialsContainer">
        ${testimonials.map((t, i) => `
          <div class="testimonial-card" data-index="${i}">
            <div class="testimonial-rating">${'★'.repeat(t.rating)}</div>
            <p class="testimonial-text">${t.text}</p>
            <div class="testimonial-author">— ${t.name}</div>
          </div>
        `).join("")}
      </div>
      <button class="testimonial-btn next" onclick="nextTestimonial()">⏩</button>
    </div>
    <div class="testimonial-dots" id="testimonialDots">
      ${testimonials.map((_, i) => `
        <span class="testimonial-dot ${i === 0 ? 'active' : ''}"
        onclick="goToTestimonial(${i})"></span>
      `).join("")}
    </div>
  `;
}

// прогресс-бары навыков
function createSkills(skills) {
  if (!skills || !skills.length) return "";

  return `
    <div class="skills-section">
      <h2>Мои навыки</h2>
      <div class="skills-container">

```



```

    ${skills.map(s => `
      <div class="skill-item">
        <div class="skill-header">
          <span class="skill-name">${s.name}</span>
          <span class="skill-percent">${s.level}%</span>
        </div>
        <div class="skill-bar">
          <div class="skill-progress" style="width: ${s.level}%></div>
        </div>
      </div>
    `).join("")}
  </div>
</div>
`;
}

```

// КОМПОНЕНТ технологий

```

function createTechStack(techs) {
  if (!techs || !techs.length) return "";

  return `
    <div class="tech-section">
      <h2>Технологии</h2>
      <div class="tech-cloud">
        ${techs.map(tech => `
          <span class="tech-tag">${tech}</span>
        `).join("")}
      </div>
    </div>
  `;
}

```

// 7. АНИМАЦИЯ ЦИФР (плавный подсчет статистики)

```

function animateNumbers() {
  const stats = document.querySelectorAll('.stat-value');
  stats.forEach(stat => {
    const target = parseInt(stat.getAttribute('data-target'));
    let current = 0;
    const increment = target / 50;
    const timer = setInterval(() => {
      current += increment;
      if (current >= target) {
        stat.textContent = target;
        clearInterval(timer);
      } else {
        stat.textContent = Math.floor(current);
      }
    }, 20);
  });
}

```

```

}

// логика слайдера
let currentTestimonial = 0;

window.prevTestimonial = function() {
  const testimonials = window.homeData?.testimonials || [];
  if (testimonials.length === 0) return;
  currentTestimonial = (currentTestimonial - 1 + testimonials.length) % testimonials.length;
  updateTestimonialDisplay();
};

window.nextTestimonial = function() {
  const testimonials = window.homeData?.testimonials || [];
  if (testimonials.length === 0) return;
  currentTestimonial = (currentTestimonial + 1) % testimonials.length;
  updateTestimonialDisplay();
};

window.goToTestimonial = function(index) {
  currentTestimonial = index;
  updateTestimonialDisplay();
};

// Обновление активного отзыва и точек слайдера
function updateTestimonialDisplay() {
  const container = document.getElementById('testimonialsContainer');
  const dots = document.querySelectorAll('.testimonial-dot');

  if (container) {
    container.style.transform = `translateX(-${currentTestimonial * 100}%)`;
  }

  dots.forEach((dot, index) => {
    if (index === currentTestimonial) {
      dot.classList.add('active');
    } else {
      dot.classList.remove('active');
    }
  });
}

// ПОСТРОЕНИЕ СТРАНИЦЫ
function buildPage(pageId) {
  let data;
  if (pageId === 'privacy-policy') {
    data = window.privacyPolicyData;
  } else if (pageId === 'terms-of-use') {
    data = window.termsOfUseData;
  }
}

```

```

    } else {
      data = window[pageId + 'Data'];
    }

    if (!data) {
      return `

# Ошибка загрузки</h1><p>Данные для страницы "${pageId}" не найден</p>`; } let html = `${data.title || ""}</h1>`; if (pageId === 'home') { html += ` <div class="home-page"> <div class="home-grid"> ${data.image ? ` <div class="home-image">  </div> ` : ""} <div class="home-content"> ${data.name ? `<h2>${data.name}</h2>` : ""} ${data.bio || ""} <!-- ===== ИСПРАВЛЕНО: используем data.skills ===== --> ${data.skills ? ` <div class="skills-section"> <h3>За эти годы я освоил:</h3> <ul class="skills-list"> ${data.skills.map(skill => `<li>${skill}</li>`).join("")} </ul> </div> ` : ""} <!-- ===== --> ${data.projectsText || ""} </div> </div> ${data.stats ? createStats(data.stats) : ""} ${data.features ? createFeatures(data.features) : ""} ${data.skillsProgress ? createSkills(data.skillsProgress) : ""} ${data.techStack ? createTechStack(data.techStack) : ""} </div> `; } else if (pageId === 'portfolio') { html += ` <div class="portfolio-page">


```

```

<div class="portfolio-grid">
  ${data.projects ? data.projects.map(project => `
    <div class="portfolio-card">
      <div class="portfolio-image">
        
      </div>
      <div class="portfolio-info">
        <h3>${project.name}</h3>
        <p class="portfolio-year">${project.year || ""}</p>
        <p class="portfolio-desc">${project.description || ""}</p>
        <div class="portfolio-tech">
          ${project.technologies ? project.technologies.map(tech =>
            `<span class="tech-badge">${tech}</span>`
          ).join(" ")}
        </div>
        <div>
          ${project.photo ? `
            <a href="${project.photo}" class="portfolio-link" target="_blank"
onclick="event.preventDefault(); window.open(this.href);">Посмотреть →</a>
          ` : ""}
        </div>
      </div>
    </div>
  `).join(" ")}
</div>

<!-- ОТЗЫВЫ КЛИЕНТОВ -->
${window.homeData?.testimonials ? `
  <div class="portfolio-testimonials">
    <h3>Что говорят клиенты</h3>
    ${renderTestimonialsInner(window.homeData.testimonials)}
  </div>
` : ""}

${data.contactText ? `
  <div class="portfolio-contact">
    <p>Хотите такой же сайт? <a href="#" onclick="window.updateContent('contacts');
return false;">Свяжитесь со мной</a> в разделе "Контакты"!</p>
  </div>
` : ""}
</div>
`;
} else if (pageId === 'services') {
  html += `
    <div class="services-page">
      <div class="services-grid">
        `;

  if (data.items && data.items.length) {
    data.items.forEach(item => {
      html += `

```

```

        <div class="service-card">
            <div class="service-icon">${item.icon || '🔧'}</div>
            <h3>${item.name}</h3>
            <div class="service-time">🕒 ${item.time}</div>
            <p class="service-description">${item.description || ""}</p>
            <div class="service-price">${item.price || 'Цена договорная'}</div>
            <button class="service-order"
onclick="window.orderService('${item.name}')">Заказать сайт</button>
        </div>
    `;
});
}

html += `
    <div class="services-disclaimer">
        <p>${data.disclaimer || ""}</p>
        <small>${data.note || ""}</small>
    </div>
</div>
`;
} else if (pageId === 'blog') {
    if (data.posts && data.posts.length) {
        html += '<div class="blog-posts">';

        data.posts.forEach((post, index) => {
            html += `
                <div class="blog-card">
                    ${post.image ? `
                        <div class="blog-card-image">
                            
                        </div>
                    ` : ""}
                    <div class="blog-card-content">
                        <div class="blog-meta">
                            <span class="blog-date">${post.date || ""}</span>
                            <span class="blog-category">${post.category || ""}</span>
                        </div>
                        <h3>${post.title || ""}</h3>
                        <p class="blog-preview">${post.preview || ""}</p>
                        <div class="blog-footer">
                            <div class="blog-stats">
                                <span>💖 ${post.likes || 0}</span>
                                <span>💬 ${post.comments || 0}</span>
                            </div>
                            <button class="read-more" onclick="window.showPost(${index +
1})">Читать →</button>
                        </div>
                    `;
            }
        });
    }
}

```

```

        </div>
    </div>
    `;
});

html += '</div>';
}
} else if (pageId === 'contacts') {
    html += `
        <div class="contacts-page">
            <div class="contacts-grid">
                <!-- Левая колонка с контактами -->
                <div class="contacts-info">
                    <h2>Свяжитесь со мной</h2>

                    <div class="contact-card">
                        <div class="contact-item">
                            <span class="contact-icon"> 👤 </span>
                            <div>
                                <strong>${data.name || " "}</strong><br>
                                <span>${data.company || " "}</span>
                            </div>
                        </div>

                        ${data.phone ? `
                            <div class="contact-item">
                                <span class="contact-icon"> ☎️ </span>
                                <div>
                                    <strong>Телефон:</strong><br>
                                    <a href="tel:${data.phone}">${data.phone}</a>
                                </div>
                            </div>
                        ` : ""}

                        ${data.email ? `
                            <div class="contact-item">
                                <span class="contact-icon"> ✉️ </span>
                                <div>
                                    <strong>Email:</strong><br>
                                    <a href="mailto:${data.email}">${data.email}</a>
                                </div>
                            </div>
                        ` : ""}

                        ${data.address ? `
                            <div class="contact-item">
                                <span class="contact-icon"> 📍 </span>
                                <div>
                                    <strong>Адрес:</strong><br>

```

```

        <span>${data.address}</span>
    </div>
</div>
`: "}"

    ${data.workHours ? `
        <div class="contact-item">
            <span class="contact-icon"> 📞 </span>
            <div>
                <strong>Часы работы:</strong><br>
                <span>${data.workHours}</span><br>
                ${data.weekend ? `<span>${data.weekend}</span>` : ""}
            </div>
        </div>
    ` : ""}
</div>

    ${data.responseTime ? `
        <div class="contact-response">
            ${data.responseTime}
        </div>
    ` : ""}

    ${data.languages ? `
        <div class="contact-languages">
            <strong>Языки:</strong> ${data.languages.join(', ')}
        </div>
    ` : ""}

    ${data.workWith ? `
        <div class="contact-work-with">
            <strong>Работаю с:</strong> ${data.workWith}
        </div>
    ` : ""}
</div>

<div class="contacts-right">
    <div class="contact-form-section">
        <h3>Напишите мне</h3>
        <form class="contact-form" id="contactForm"
onsubmit="handleFormSubmit(event)">
            <input type="text" placeholder="Ваше имя" required>
            <input type="email" placeholder="Ваш email" required>
            <input type="text" placeholder="Тема">
            <textarea placeholder="Ваше сообщение" rows="5" required></textarea>
            <button type="submit" class="submit-btn">Отправить
сообщение</button>
        </form>
    </div>

```

```

    ${data.social ? `
      <div class="contact-social">
        <h3>Я в соцсетях</h3>
        <div class="social-grid">
          ${data.social.map(s => `
            <a href="${s.url}" target="_blank" class="social-contact-card">
              <span class="social-contact-icon">${s.icon}</span>
              <div class="social-contact-info">
                <strong>${s.name}</strong>
                <small>${s.username || ""}</small>
              </div>
            </a>
          `).join("")}
        </div>
      </div>
    ` : ""}
  </div>

  <div class="contact-map">
    <h3>Как нас найти</h3>
    <div class="map-placeholder"
      onclick="window.open('https://yandex.ru/maps/?ll=${data.coordinates.lng},${data.coordinates.lat}&z=17&pt=${data.coordinates.lng},${data.coordinates.lat},pm2blm', '_blank')">
      <div class="map-overlay">
        <span>🗺 Открыть карту</span>
        <small>${data.address || 'Нажмите чтобы открыть'}</small>
      </div>
    </div>
  </div>
</div>
`;
} else if (pageId === 'privacy-policy') {
  console.log('👉 СТРОИМ privacy-policy, data:', data);
  html += `
    <div class="legal-page">
      <div class="legal-content">${data.content}</div>
      <button class="nav-btn" onclick="window.updateContent('home')">← На
главную</button>
    </div>
  `;
} else if (pageId === 'terms-of-use') {
  console.log('👉 СТРОИМ terms-of-use, data:', data);
  html += `
    <div class="legal-page">
      <div class="legal-content">${data.content}</div>
      <button class="nav-btn" onclick="window.updateContent('home')">← На
главную</button>
    </div>
  `;
}

```



```

    `;
}

return `<div class="page">${html}</div>`;
}

// ОБНОВЛЕНИЕ КОНТЕНТА
window.updateContent = function(pageId) {
    const content = document.getElementById('content');
    if (content) {
        content.innerHTML = buildPage(pageId);

        if (pageId === 'home') {
            setTimeout(animateNumbers, 500);
        }
    }
};

// ФУНКЦИЯ ДЛЯ ПОКАЗА ОТДЕЛЬНОГО ПОСТА БЛОГА
window.showPost = function(postId) {
    console.log('Открываем пост:', postId);

    // Получение данных конкретного поста
    const post = window['post' + postId + 'Data'];

    if (!post) {
        alert('Пост не найден!');
        return;
    }

    const postHtml = `
        <div class="post-page">
            <button class="back-btn" onclick="window.updateContent('blog')">← Назад к
            блогу</button>

            <article class="post-article">
                <h1>${post.title}</h1>

                <div class="post-meta">
                    <span>📅 ${post.date}</span>
                    <span>🏷️ ${post.category}</span>
                    <span>👤 ${post.author}</span>
                </div>

                ${post.image ? `` :
            `
                <div class="post-content">
                    ${post.content}

```

```

</div>

${post.tags ? `
  <div class="post-tags">
    ${post.tags.map(tag => `<span class="post-tag">#${tag}</span>`).join("")}
  </div>
  ` : ""}

<div class="post-stats">
  <span>❤️ ${post.likes} лайков</span>
  <span>💬 ${post.comments} комментариев</span>
</div>

<div class="post-navigation">
  <button class="nav-btn" onclick="window.updateContent('blog')">← Все
статьи</button>
  <button class="nav-btn" id="shareBtn">📄 Поделиться</button>
</div>
</article>
</div>
`;

// ПОСТ В КОНТЕНТ
document.getElementById('content').innerHTML = postHtml;

// ОБРАБОТЧИК ДЛЯ КНОПКИ "ПОДЕЛИТЬСЯ"
setTimeout(() => {
  const shareBtn = document.getElementById('shareBtn');
  if (shareBtn) {
    shareBtn.onclick = async () => {
      const shareData = {
        title: post.title,
        text: post.preview || post.content.replace(/<[>]*>/g, "").substring(0, 100) + '...',
        url: window.location.href
      };

      // Web Share API (работает на телефонах)
      if (navigator.share) {
        try {
          await navigator.share(shareData);
          console.log('Успешно поделились!');
          return;
        } catch (err) {
          if (err.name !== 'AbortError') {
            console.error('Ошибка шаринга:', err);
            fallbackShare(shareData);
          }
        }
      }
    }
  }
} else {

```

```

        // Если Web Share не поддерживается, то fallback
        fallbackShare(shareData);
    }
};
}, 100);

window.scrollTo({ top: 0, behavior: 'smooth' });
};

// если Web Share API не поддерживается или упал
function fallbackShare(shareData) {
    // модальное окно с выбором действия
    const action = confirm(
        `Поделиться статьей:\n\n` +
        `Заголовок: ${shareData.title}\n\n` +
        `Нажмите ОК, чтобы скопировать ссылку, или Отмена, чтобы поделиться в Telegram`
    );

    if (action) {
        navigator.clipboard.writeText(shareData.url).then(() => {
            alert(`☑ Ссылка скопирована!\n\nТеперь вы можете вставить её куда угодно.`);
        }).catch(() => {
            prompt('Скопируйте ссылку:', shareData.url);
        });
    } else {
        const telegramUrl =
`https://t.me/share/url?url=${encodeURIComponent(shareData.url)}&text=${encodeURIComponent(shareData.title)}`;
        window.open(telegramUrl, '_blank', 'width=600,height=400');
    }
}

// переопределение функции updateContent, чтобы она работала и с post-* ID
const originalUpdateContent = window.updateContent;
window.updateContent = function(pageId) {
    console.log('👉 updateContent (новая) вызвана для:', pageId);

    if (pageId === 'blog') {
        originalUpdateContent('blog');
    } else if (pageId.startsWith('post-')) {
        const postId = pageId.replace('post-', '');
        window.showPost(postId);
    } else {
        originalUpdateContent(pageId);
    }
};
console.log('☑ updateContent переопределена');

```

```
// ОТПРАВКА ФОРМЫ В TELEGRAM (через Bot API)
window.handleFormSubmit = async function(event) {
  event.preventDefault();

  // данные
  const form = event.target;
  const name = form.querySelector('input[placeholder="Ваше имя"]').value;
  const email = form.querySelector('input[placeholder="Ваш email"]').value;
  const subject = form.querySelector('input[placeholder="Тема"]').value;
  const message = form.querySelector('textarea').value;

  const TOKEN = '8762054263:AAGsA160X--1m_md14URdaDoe4ZeCE6IZQo';
  const CHAT_ID = '631194715';

  let text = `
    📄 НОВОЕ СООБЩЕНИЕ С САЙТА
    _____

    👤 Имя: ${name}
    ✉ Email: ${email}
    📧 Тема: ${subject || 'не указана'}
    _____

    💬 Сообщение:
    ${message}
    _____

    🕒 Время: ${new Date().toLocaleString('ru-RU')}
  `.trim();

  if (text.length > 4090) {
    text = text.slice(0, 4087) + '...';
  }

  const actionUrl = `https://api.telegram.org/bot${TOKEN}/sendMessage`;

  // 1) fetch — если среда не режет CORS
  try {
    const response = await fetch(actionUrl, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ chat_id: CHAT_ID, text: text })
    });
    const data = await response.json().catch(() => ({}));
    if (response.ok && data.ok) {
      alert('✅ Спасибо, ${name}! Сообщение отправлено в Telegram. \nЯ отвечу вам на ${email} в ближайшее время.`);
      form.reset();
      return;
    }
    if (data.description) {
      console.error('Telegram API:', data);
    }
  }

```

```

        alert(`❌ Ошибка Telegram: ${data.description}\n\nПроверь токен, Chat ID и что ты
написал боту /start.`);
        return;
    }
} catch (e) {
    console.warn('fetch к Telegram не сработал (часто CORS) — пробуем отправку через
iframe:', e);
}

// 2) Fallback: POST формой в скрытый iframe (CORS на ответ не мешает доставке
запроса)
let iframe = document.getElementById('telegramSubmitFrame');
if (!iframe) {
    iframe = document.createElement('iframe');
    iframe.name = 'telegramSubmitFrame';
    iframe.id = 'telegramSubmitFrame';
    iframe.title = 'Telegram';
    iframe.setAttribute('aria-hidden', 'true');
    iframe.style.cssText = 'width:0;height:0;border:0;visibility:hidden;position:absolute';
    document.body.appendChild(iframe);
}

const proxyForm = document.createElement('form');
proxyForm.method = 'POST';
proxyForm.action = actionUrl;
proxyForm.target = 'telegramSubmitFrame';
proxyForm.acceptCharset = 'UTF-8';
proxyForm.style.display = 'none';

const addField = (n, v) => {
    const inp = document.createElement('input');
    inp.type = 'hidden';
    inp.name = n;
    inp.value = v;
    proxyForm.appendChild(inp);
};
addField('chat_id', String(CHAT_ID));
addField('text', text);

document.body.appendChild(proxyForm);
proxyForm.submit();
setTimeout(() => proxyForm.remove(), 2000);

alert(
    `✅ Запрос отправлен.\n\nЕсли сообщения нет в Telegram через минуту — проверь
токен в script.js, Chat ID и что ты написал боту /start.`
);
form.reset();
};

```

```

// ФУНКЦИЯ ДЛЯ ЗАПОЛНЕНИЯ ФОРМЫ ПРИ ЗАКАЗЕ УСЛУГИ
window.orderService = function(serviceName) {
  window.updateContent('contacts');

  setTimeout(() => {
    const subjectInput = document.querySelector('input[placeholder="Тема"]');
    if (subjectInput) {
      subjectInput.value = `Заказ услуги: ${serviceName}`;
      // подсветка поля
      subjectInput.style.borderColor = '#667eea';
      subjectInput.style.boxShadow = '0 0 0 3px rgba(102, 126, 234, 0.1)';

      // минус подсветка через 2 секунды
      setTimeout(() => {
        subjectInput.style.borderColor = "";
        subjectInput.style.boxShadow = "";
      }, 2000);
    }
  }, 300);
};

// инициализация
document.addEventListener('DOMContentLoaded', function() {
  window.updateContent('home');
});

```